

# Unit 3.6: Multivariate Random Variable Generation

Dr. Manoj C. Patil

2026-09-09

## Learning Objectives

By the end of this unit, a student should be able to

- explain why generating a random vector generally requires more than generating each coordinate separately, and how dependence between coordinates must be built into the generating mechanism,
- state and prove the correctness of the sequential conditional distribution method for simulating a bivariate or multivariate random vector,
- derive the linear (Cholesky-based) representation of a Normal random vector with a prescribed covariance matrix, and prove its correctness,
- implement both the sequential conditional method and the Cholesky method in R, for bivariate and higher-dimensional targets,
- relate the sequential conditional method to Gibbs sampling, and understand the distinction between the two,
- evaluate the strengths and limitations of each multivariate generation strategy in applied settings.

## Prerequisites

A reader should already be comfortable with

- the inverse transformation, acceptance-rejection, and distributional-relationship methods for univariate generation (Units 3.3-3.5),
- the definition of a joint density and of a conditional density, and the multiplication rule  $f(x, y) = f_X(x) f_{Y|X}(y | x)$ ,
- basic matrix algebra, including symmetric positive definite matrices and the notion of a triangular matrix,
- the multivariate Normal distribution and its moment generating function, at least at the level of a first course in mathematical statistics.

## Introduction

Every method developed so far in this course generates a single number at a time. Many applications, however, require a *vector* of numbers that are not independent of one another: the systolic and diastolic blood pressure of a patient, the returns of several assets in a portfolio on a given day, or the coordinates of a point at which several structural loads act simultaneously. Generating each coordinate on its own, using the univariate methods of Units 3.3-3.5, would produce a vector with the correct marginal behaviour in each coordinate but with no dependence between coordinates whatsoever — a serious flaw whenever the dependence itself is the feature of scientific interest, as it usually is.

The mathematical resolution is old and elegant. Any joint density can be factored, by repeated application of the definition of conditional probability, into a product of a marginal density and a chain of conditional densities — a fact usually first encountered as the “multiplication rule” or “chain rule” of probability. This

factorization turns the problem of generating a dependent vector into a sequence of ordinary univariate generation problems: simulate the first coordinate from its marginal distribution, then simulate the second coordinate from its distribution *conditional on* the value just simulated for the first, and so on. When the target vector is Gaussian, an even more economical route is available: any Normal vector with a prescribed covariance matrix can be built directly by a single linear transformation of independent standard Normal variables, using the Cholesky decomposition of the covariance matrix — a technique introduced by André-Louis Cholesky in the early twentieth century for solving systems of normal equations in geodesy, and now a standard tool throughout computational statistics.

The practical importance of multivariate generation is difficult to overstate. Simulating correlated asset returns is the foundation of portfolio risk analysis and derivative pricing in finance; simulating correlated component lifetimes is essential in reliability engineering whenever components share a common operating environment; and multivariate posterior simulation, whether by direct construction or by the iterative Gibbs sampling algorithm introduced at the end of this unit, is a central tool of modern Bayesian statistics. This unit develops both the general sequential conditional method and its specialization to the Normal distribution via the Cholesky decomposition.

## Intuition

Suppose the goal is to simulate a pair of correlated measurements, such as a student’s midterm score and final exam score, where a higher midterm score tends to be associated with a higher final score. Simulating the two scores completely independently would fail to capture this association at all. A natural fix suggests itself: first simulate the midterm score from its own (marginal) distribution, observe the value obtained, and then simulate the final score from a distribution that has been *shifted* to reflect the midterm value just drawn — a higher midterm value should shift the distribution of the final score upward. This two-step recipe — draw the first coordinate, then draw the second coordinate from a distribution that depends on the first — is exactly the sequential conditional method, and it generalizes immediately to three, four, or any number of coordinates, each drawn conditional on all those drawn before it.

For the particular case of Normally distributed measurements, there is a second, purely linear-algebraic way of seeing the same idea. Two independent standard Normal quantities, call them  $W_1$  and  $W_2$ , carry no association with each other. Forming a new pair  $X_1 = W_1$  and  $X_2 = \rho W_1 + \sqrt{1 - \rho^2} W_2$  blends a fraction  $\rho$  of the first quantity into the second; the larger  $\rho$  is, the more the two resulting quantities move together. This blending construction is precisely what the Cholesky decomposition automates in arbitrary dimension: it finds the “recipe” of linear blends, one per coordinate, that turns a batch of independent standard Normal quantities into a Normal vector with any desired covariance structure.

## Formal Definitions

### Random Vectors and Joint Densities

**Definition.** A **random vector**  $\mathbf{X} = (X_1, \dots, X_p)'$  is a collection of  $p$  random variables considered jointly, with joint density (or joint probability mass function, in the discrete case)  $f(x_1, \dots, x_p)$ . A **bivariate distribution** is the special case  $p = 2$ .

### Conditional Densities and the Chain Rule

**Definition.** For a joint density  $f(x_1, \dots, x_p)$ , the **conditional density** of  $X_i$  given  $X_1 = x_1, \dots, X_{i-1} = x_{i-1}$  is

$$f_i(x_i \mid x_1, \dots, x_{i-1}) = \frac{f(x_1, \dots, x_i)}{f(x_1, \dots, x_{i-1})},$$

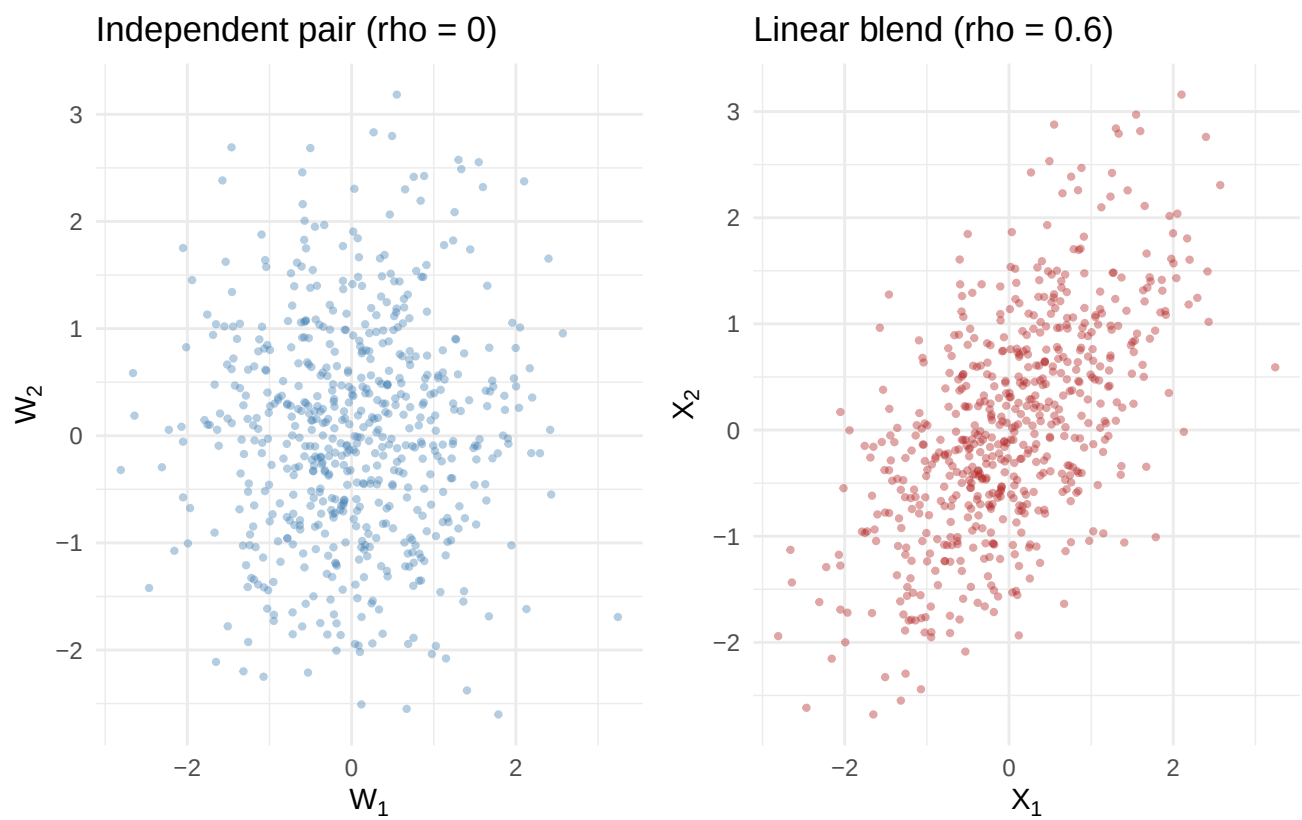


Figure 1: Independent standard Normal pairs (left) versus a linear blend with  $\rho = 0.6$  (right); the blend introduces visible positive association without changing either marginal distribution.

wherever the denominator is positive, with  $f(x_1, \dots, x_i)$  denoting the joint density of the first  $i$  coordinates. Repeated application of this definition gives the **chain rule factorization**

$$f(x_1, \dots, x_p) = f_1(x_1) f_2(x_2 | x_1) f_3(x_3 | x_1, x_2) \cdots f_p(x_p | x_1, \dots, x_{p-1}), \quad (6.1)$$

where  $f_1$  is the marginal density of  $X_1$  alone.

where

- $f_1(x_1)$  denotes the marginal density of the first coordinate,
- $f_i(x_i | x_1, \dots, x_{i-1})$  denotes the density of the  $i$ -th coordinate conditional on the values of all preceding coordinates,
- the ordering of coordinates  $1, \dots, p$  is arbitrary; any ordering yields a valid factorization of the same joint density, though the individual conditional densities differ across orderings.

## The Multivariate Normal Distribution

**Definition.** A random vector  $\mathbf{X} = (X_1, \dots, X_p)'$  has the **multivariate Normal distribution** with mean vector  $\boldsymbol{\mu}$  and covariance matrix  $\boldsymbol{\Sigma}$  (symmetric and positive definite), written  $\mathbf{X} \sim N_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ , if its moment generating function is

$$M_{\mathbf{X}}(\mathbf{t}) = \exp\left(\mathbf{t}'\boldsymbol{\mu} + \frac{1}{2}\mathbf{t}'\boldsymbol{\Sigma}\mathbf{t}\right), \quad \mathbf{t} \in \mathbb{R}^p. \quad (6.2)$$

**Definition (Cholesky decomposition).** For a symmetric positive definite matrix  $\boldsymbol{\Sigma}$ , the **Cholesky decomposition** is the unique factorization  $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}'$ , where  $\mathbf{L}$  is lower triangular with strictly positive diagonal entries.

## Theorems

### Theorem 6.1: Correctness of the Sequential Conditional Method

**Statement.** Let  $f(x_1, \dots, x_p)$  admit the chain rule factorization (6.1). If  $X_1$  is drawn from  $f_1$ , and for  $i = 2, \dots, p$ ,  $X_i$  is drawn from  $f_i(\cdot | X_1, \dots, X_{i-1})$  using the values already drawn for  $X_1, \dots, X_{i-1}$ , then the resulting vector  $(X_1, \dots, X_p)$  has joint density  $f(x_1, \dots, x_p)$ .

**Assumptions.** Each conditional density  $f_i(\cdot | x_1, \dots, x_{i-1})$  is a valid density (or pmf) for every value of the conditioning arguments actually reachable by the construction.

**Proof.** By induction on  $p$ . For  $p = 1$ , the claim is immediate:  $X_1 \sim f_1$  by construction. Assume the claim holds for  $p - 1$  coordinates, so that  $(X_1, \dots, X_{p-1})$ , generated sequentially, has joint density  $f(x_1, \dots, x_{p-1})$ . By construction,  $X_p$  is drawn, conditional on  $(X_1, \dots, X_{p-1}) = (x_1, \dots, x_{p-1})$ , from  $f_p(\cdot | x_1, \dots, x_{p-1})$ ; by the definition of a conditional density, this means the joint density of  $(X_1, \dots, X_p)$  satisfies

$$f(x_1, \dots, x_p) = f(x_1, \dots, x_{p-1}) f_p(x_p | x_1, \dots, x_{p-1}),$$

which is exactly the chain rule factorization (6.1) for  $p$  coordinates. By the induction hypothesis,  $f(x_1, \dots, x_{p-1})$  is correct, so  $(X_1, \dots, X_p)$  has joint density  $f(x_1, \dots, x_p)$ , completing the induction. ■

**Remarks.** The theorem places no restriction on the ordering of coordinates: any ordering of  $X_1, \dots, X_p$  yields a valid factorization and a correct algorithm, although the individual conditional densities required will generally differ across orderings, and some orderings may make those conditionals far easier (or far harder) to derive and simulate than others. This method should not be confused with **Gibbs sampling**, an iterative Markov chain Monte Carlo technique that repeatedly cycles through the *full* conditional distributions  $f(x_i | x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_p)$  (conditioning on *all other* coordinates, not only the preceding ones) and converges only asymptotically to the joint distribution. Theorem 6.1 describes a single-pass, exact method

that requires the successive conditionals of the chain rule; Gibbs sampling is used precisely in settings where those successive conditionals are unavailable in closed form but the full conditionals are.

**Examples.** Full worked examples are given in the Numerical Examples section below, for both a continuous and a discrete bivariate target.

**Applications.** The sequential conditional method underlies simulation from graphical models and Bayesian networks, in which the joint distribution is specified natively as a product of conditional distributions along a directed graph, and is the exact-sampling ancestor of the approximate Gibbs sampling algorithm used throughout Bayesian computation.

## Theorem 6.2: The Cholesky Construction of a Multivariate Normal Vector

**Statement.** Let  $\mathbf{Z} = (Z_1, \dots, Z_p)'$  have independent standard Normal components, so  $\mathbf{Z} \sim N_p(\mathbf{0}, \mathbf{I}_p)$ . Let  $\Sigma$  be symmetric positive definite with Cholesky decomposition  $\Sigma = \mathbf{L}\mathbf{L}'$ . Then

$$\mathbf{X} = \boldsymbol{\mu} + \mathbf{L}\mathbf{Z} \sim N_p(\boldsymbol{\mu}, \Sigma). \quad (6.3)$$

**Assumptions.**  $\Sigma$  is symmetric positive definite, guaranteeing that the Cholesky decomposition exists and is unique with a strictly positive diagonal.

**Proof.** Since  $\mathbf{Z} \sim N_p(\mathbf{0}, \mathbf{I}_p)$ , its moment generating function is  $M_{\mathbf{Z}}(\mathbf{t}) = \exp(\mathbf{t}'\mathbf{t}/2)$  (the product of  $p$  independent standard Normal moment generating functions). For  $\mathbf{X} = \boldsymbol{\mu} + \mathbf{L}\mathbf{Z}$ ,

$$\begin{aligned} M_{\mathbf{X}}(\mathbf{t}) &= E[\exp(\mathbf{t}'\mathbf{X})] = E[\exp(\mathbf{t}'\boldsymbol{\mu} + \mathbf{t}'\mathbf{L}\mathbf{Z})] \\ &= \exp(\mathbf{t}'\boldsymbol{\mu}) E[\exp((\mathbf{L}'\mathbf{t})'\mathbf{Z})] = \exp(\mathbf{t}'\boldsymbol{\mu}) M_{\mathbf{Z}}(\mathbf{L}'\mathbf{t}) \\ &= \exp(\mathbf{t}'\boldsymbol{\mu}) \exp\left(\frac{1}{2}(\mathbf{L}'\mathbf{t})'(\mathbf{L}'\mathbf{t})\right) = \exp\left(\mathbf{t}'\boldsymbol{\mu} + \frac{1}{2}\mathbf{t}'\mathbf{L}\mathbf{L}'\mathbf{t}\right) \\ &= \exp\left(\mathbf{t}'\boldsymbol{\mu} + \frac{1}{2}\mathbf{t}'\Sigma\mathbf{t}\right), \end{aligned}$$

using  $\mathbf{L}\mathbf{L}' = \Sigma$  in the last step. This is exactly the moment generating function (6.2) of  $N_p(\boldsymbol{\mu}, \Sigma)$ , and since the moment generating function determines the distribution uniquely,  $\mathbf{X} \sim N_p(\boldsymbol{\mu}, \Sigma)$ . ■

**Remarks.** The bivariate blending construction of the Intuition section,  $X_1 = W_1$ ,  $X_2 = \rho W_1 + \sqrt{1 - \rho^2} W_2$ , is precisely the  $p = 2$  special case of (6.3) with correlation matrix

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \quad \text{and Cholesky factor} \quad \mathbf{L} = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1 - \rho^2} \end{pmatrix},$$

as verified directly by matrix multiplication:

$$\mathbf{L}\mathbf{L}' = \begin{pmatrix} 1 & \rho \\ \rho & \rho^2 + 1 - \rho^2 \end{pmatrix} = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}.$$

**Examples.** Worked numerically, for both  $p = 2$  and  $p = 3$ , in the Numerical Examples section.

**Applications.** The Cholesky construction is the standard technique for simulating correlated asset returns in quantitative finance, correlated random effects in multilevel models, and correlated structural loads in engineering reliability analysis.

## Mathematical Derivations

### Equivalence of the Conditional Method and the Cholesky Method for the Normal Distribution

For a Normal vector, the general chain-rule factorization of Theorem 6.1 and the Cholesky construction of Theorem 6.2 are not merely two unrelated techniques; they compute exactly the same generating recipe. The rows of the Cholesky factor  $\mathbf{L}$  are precisely the coefficients of the successive conditional-mean regressions implied by (6.1): writing  $\mathbf{X} = \boldsymbol{\mu} + \mathbf{L}\mathbf{Z}$  coordinate by coordinate,

$$X_i = \mu_i + \sum_{j=1}^i L_{ij}Z_j = \left(\mu_i + \sum_{j=1}^{i-1} L_{ij}Z_j\right) + L_{ii}Z_i, \quad i = 1, \dots, p,$$

shows that, conditional on  $X_1, \dots, X_{i-1}$  (equivalently, on  $Z_1, \dots, Z_{i-1}$ , since the map between them is triangular and invertible),  $X_i$  is Normal with mean equal to the linear regression of  $X_i$  on  $X_1, \dots, X_{i-1}$  and variance  $L_{ii}^2$ . This is exactly the conditional density  $f_i(x_i | x_1, \dots, x_{i-1})$  required by (6.1), specialized to the Gaussian case. In other words, for the Normal distribution, sequentially applying Theorem 6.1 with Normal conditional distributions and applying Theorem 6.2 via the Cholesky decomposition are two descriptions of the identical algorithm; the Cholesky decomposition is simply the closed-form, matrix-algebraic solution of the successive linear regressions that the conditional method would otherwise require deriving by hand, one coordinate at a time (Pourahmadi, 1999).

## Algorithms

### Algorithm 6.1: General Sequential Conditional Distribution Method

**Purpose.** Generate a random vector  $(X_1, \dots, X_p)$  with a specified joint distribution, given the marginal density of  $X_1$  and the successive conditional densities implied by the chain rule (6.1).

**Input.** Marginal sampler for  $X_1$ ; conditional samplers for  $X_i | X_1, \dots, X_{i-1}$ ,  $i = 2, \dots, p$ ; required sample size  $n$ .

**Output.** A matrix of  $n$  simulated vectors, each of dimension  $p$ .

**Step-by-step procedure.**

1. For each of  $n$  replications:
  - (a) draw  $x_1$  from the marginal density of  $X_1$ ;
  - (b) for  $i = 2, \dots, p$ , draw  $x_i$  from the conditional density of  $X_i$  given the values  $x_1, \dots, x_{i-1}$  already drawn;
2. Return the collection of  $n$  vectors  $(x_1, \dots, x_p)$ .

**Pseudo-code.**

Algorithm: Sequential Conditional Method

Input: marginal sampler for  $X_1$ , conditional samplers for  $X_2..X_p$ ,  $n$

Output:  $n \times p$  matrix of simulated vectors

1. For  $k = 1$  to  $n$ :
  - $x[1] \leftarrow$  draw from marginal of  $X_1$
  - For  $i = 2$  to  $p$ :
    - $x[i] \leftarrow$  draw from conditional of  $X_i$  given  $x[1..i-1]$
  - Store  $x$  as row  $k$  of output
2. Return output

**Flowchart description.** Start; draw the first coordinate from its marginal distribution; for each remaining coordinate, in order, draw from its distribution conditional on all coordinates already drawn; after all  $p$  coordinates are drawn, record the completed vector; repeat for  $n$  replications; end.

**Complexity.**  $O(np)$  time overall, provided each individual conditional draw costs  $O(1)$ ; the memory required to store the output is  $O(np)$ .

**Practical considerations.** The choice of ordering among the  $p$  coordinates does not affect correctness (Theorem 6.1) but can greatly affect how tractable the required conditional densities are; an ordering that produces simple, standard conditional distributions (such as the Normal-Normal structure of Section 7) should always be preferred when one is available.

## Algorithm 6.2: Bivariate Normal Generation via Conditional Distributions

**Purpose.** Generate pairs  $(X_1, X_2)$  from the standard bivariate Normal distribution with correlation  $\rho$ , using the sequential conditional method specialized to the Normal-Normal case.

**Input.** Correlation  $\rho \in (-1, 1)$ ; sample size  $n$ .

**Output.** An  $n \times 2$  matrix of simulated pairs.

**Step-by-step procedure.**

1. For  $k = 1, \dots, n$ : draw  $w_1 \sim N(0, 1)$  and  $w_2 \sim N(0, 1)$  independently; set  $x_1 = w_1$  and  $x_2 = \rho w_1 + \sqrt{1 - \rho^2} w_2$ .
2. Return the  $n$  pairs  $(x_1, x_2)$ .

**Pseudo-code.**

Algorithm: Bivariate Normal, Conditional Method

Input: rho, n

Output: n x 2 matrix

1. For k = 1 to n:
  - w1 <- draw from N(0,1)
  - w2 <- draw from N(0,1)
  - x1 <- w1
  - x2 <- rho \* w1 + sqrt(1 - rho^2) \* w2
2. Return matrix of (x1, x2) pairs

**Flowchart description.** Start; draw two independent standard Normal variates; report the first unchanged as  $X_1$ ; blend the two variates using the correlation coefficient to obtain  $X_2$ ; repeat for  $n$  replications; end.

**Complexity.**  $O(n)$ , requiring two Normal draws per replication.

**Practical considerations.** The formula requires  $\rho \in (-1, 1)$  strictly; at  $\rho = \pm 1$  the conditional variance  $1 - \rho^2$  vanishes and  $X_2$  becomes a deterministic (degenerate) function of  $X_1$ , which is mathematically valid but numerically edge-case behaviour that should be handled explicitly if  $|\rho| = 1$  is a possibility in an application.

## Algorithm 6.3: Multivariate Normal Generation via Cholesky Decomposition

**Purpose.** Generate vectors from  $N_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$  for general dimension  $p$ , using Theorem 6.2.

**Input.** Mean vector  $\boldsymbol{\mu}$ ; symmetric positive definite covariance matrix  $\boldsymbol{\Sigma}$ ; sample size  $n$ .

**Output.** An  $n \times p$  matrix of simulated vectors.

**Step-by-step procedure.**

1. Compute the Cholesky factor  $\mathbf{L}$  satisfying  $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}'$ .
2. For  $k = 1, \dots, n$ : draw  $\mathbf{z} \sim N_p(\mathbf{0}, \mathbf{I}_p)$  (i.e.  $p$  independent standard Normal draws); set  $\mathbf{x} = \boldsymbol{\mu} + \mathbf{L}\mathbf{z}$ .
3. Return the  $n$  vectors  $\mathbf{x}$ .

### Pseudo-code.

Algorithm: Multivariate Normal, Cholesky Method

Input:  $\mu$  (length  $p$ ),  $\Sigma$  ( $p \times p$ ),  $n$

Output:  $n \times p$  matrix

1.  $L \leftarrow$  Cholesky factor of  $\Sigma$ , with  $\Sigma = L L'$
2. For  $k = 1$  to  $n$ :  
     $z \leftarrow p$  independent draws from  $N(0,1)$   
     $x \leftarrow \mu + L \%*\% z$
3. Return matrix of  $x$  vectors

**Flowchart description.** Start; compute the Cholesky factor of the target covariance matrix once; for each replication, draw a vector of independent standard Normal variates and apply the fixed linear transformation  $\mu + Lz$ ; repeat for  $n$  replications; end.

**Complexity.** The one-time Cholesky decomposition costs  $O(p^3)$ ; each subsequent replication costs  $O(p^2)$  for the matrix-vector multiplication, so generating  $n$  vectors costs  $O(p^3 + np^2)$  overall.

**Practical considerations.**  $\Sigma$  must be symmetric positive definite for the Cholesky decomposition to exist; in R, the built-in `chol()` function raises an informative error if the supplied matrix is not positive definite, which is a useful validation step before relying on a covariance matrix assembled by hand from individually plausible pairwise correlations that may not be jointly consistent.

## R Programming

```
set.seed(123)

#' Generate bivariate Normal pairs via the conditional distribution method
#'
#' Purpose: implements Algorithm 6.2, blending two independent standard
#' Normal draws to obtain a correlated pair.
#'
#' Arguments:
#'   sample_size  number of pairs to generate
#'   correlation   the target correlation rho, in (-1, 1)
#'
#' Returns:
#'   a matrix with sample_size rows and 2 columns
#'
#' Example:
#'   generate_bivariate_normal(sample_size = 5, correlation = 0.6)
generate_bivariate_normal <- function(sample_size, correlation) {
  first_component <- rnorm(sample_size)
  independent_component <- rnorm(sample_size)

  second_component <- correlation * first_component +
    sqrt(1 - correlation^2) * independent_component

  cbind(x1 = first_component, x2 = second_component)
}

#' Generate multivariate Normal vectors via the Cholesky decomposition
#'
#' Purpose: implements Algorithm 6.3 for a general covariance matrix.
```

```

#'
#' Arguments:
#'   sample_size    number of vectors to generate
#'   mean_vector     numeric vector of length p
#'   covariance_matrix  p x p symmetric positive definite matrix
#'
#' Returns:
#'   a matrix with sample_size rows and length(mean_vector) columns
#'
#' Example:
#'   generate_multivariate_normal(
#'     sample_size = 5,
#'     mean_vector = c(0, 0, 0),
#'     covariance_matrix = diag(3)
#'   )
generate_multivariate_normal <- function(sample_size, mean_vector, covariance_matrix) {
  dimension <- length(mean_vector)

  # R's chol() returns the upper-triangular factor U with t(U) %*% U = Sigma;
  # the lower-triangular L used in Theorem 6.2 is L = t(U)
  cholesky_lower <- t(chol(covariance_matrix))

  standard_normal_matrix <- matrix(
    rnorm(sample_size * dimension), nrow = dimension, ncol = sample_size
  )

  transformed <- mean_vector + cholesky_lower %*% standard_normal_matrix

  t(transformed)
}

#' Generate a bivariate (Exponential, conditional Uniform) pair
#'
#' Purpose: demonstrates Algorithm 6.1 for a non-Gaussian bivariate
#' target,  $X \sim \text{Exponential}(\text{rate})$ ,  $Y \mid X = x \sim \text{Uniform}(0, x)$ .
#'
#' Arguments:
#'   sample_size    number of pairs to generate
#'   rate            rate parameter of the Exponential marginal
#'
#' Returns:
#'   a matrix with sample_size rows and 2 columns
#'
#' Example:
#'   generate_exponential_uniform_pair(sample_size = 5, rate = 1)
generate_exponential_uniform_pair <- function(sample_size, rate) {
  x_values <- rexp(sample_size, rate = rate)
  y_values <- runif(sample_size, min = 0, max = x_values)

  cbind(x = x_values, y = y_values)
}

```

## Simulation Studies

### Bivariate Normal via the Conditional Method

```
set.seed(123)

bivariate_sample <- generate_bivariate_normal(sample_size = 20000, correlation = 0.6)

tibble(
  Quantity = c("Target correlation", "Empirical correlation",
               "Empirical mean X1", "Empirical mean X2",
               "Empirical variance X1", "Empirical variance X2"),
  Value = c(
    0.6, cor(bivariate_sample[, "x1"], bivariate_sample[, "x2"]),
    mean(bivariate_sample[, "x1"]), mean(bivariate_sample[, "x2"]),
    var(bivariate_sample[, "x1"]), var(bivariate_sample[, "x2"])
  )
) %>%
kable(digits = 4, caption = "Bivariate Normal via conditional method: theory vs simulation")
```

Table 1: Bivariate Normal via conditional method: theory vs simulation

| Quantity              | Value   |
|-----------------------|---------|
| Target correlation    | 0.6000  |
| Empirical correlation | 0.6034  |
| Empirical mean X1     | -0.0057 |
| Empirical mean X2     | -0.0084 |
| Empirical variance X1 | 1.0002  |
| Empirical variance X2 | 1.0061  |

### Trivariate Normal via the Cholesky Method

```
set.seed(123)

target_mean <- c(0, 0, 0)
target_covariance <- matrix(
  c(1.0, 0.5, 0.3,
    0.5, 1.0, 0.4,
    0.3, 0.4, 1.0),
  nrow = 3, byrow = TRUE
)

trivariate_sample <- generate_multivariate_normal(
  sample_size = 20000, mean_vector = target_mean, covariance_matrix = target_covariance
)

empirical_covariance <- cov(trivariate_sample)

kable(target_covariance, digits = 3, caption = "Target covariance matrix")
```

Table 2: Target covariance matrix

|     |     |     |
|-----|-----|-----|
| 1.0 | 0.5 | 0.3 |
| 0.5 | 1.0 | 0.4 |
| 0.3 | 0.4 | 1.0 |

```
kable(empirical_covariance, digits = 3, caption = "Empirical covariance matrix from 20000 simulated vec
```

Table 3: Empirical covariance matrix from 20000 simulated vectors

|       |       |       |
|-------|-------|-------|
| 1.010 | 0.509 | 0.304 |
| 0.509 | 1.014 | 0.402 |
| 0.304 | 0.402 | 0.993 |

## Non-Gaussian Bivariate Target: Exponential-Conditional-Uniform

```
set.seed(123)

exp_uniform_sample <- generate_exponential_uniform_pair(sample_size = 20000, rate = 1)

ks_marginal_x <- ks.test(exp_uniform_sample[, "x"], "pexp", rate = 1)

tibble(
  Quantity = c("KS test p-value, X marginal vs Exponential(1)",
               "Proportion of pairs with Y <= X (should be 1)"),
  Value = c(ks_marginal_x$p.value, mean(exp_uniform_sample[, "y"] <= exp_uniform_sample[, "x"]))
) %>%
  kable(digits = 4, caption = "Exponential-Uniform bivariate construction: diagnostics")
```

Table 4: Exponential-Uniform bivariate construction: diagnostics

| Quantity                                      | Value  |
|-----------------------------------------------|--------|
| KS test p-value, X marginal vs Exponential(1) | 0.2546 |
| Proportion of pairs with Y <= X (should be 1) | 1.0000 |

## Graphs

```
set.seed(123)

grid_vals <- seq(-4, 4, length.out = 120)
grid_df <- expand.grid(x1 = grid_vals, x2 = grid_vals)

bivariate_normal_density <- function(x1, x2, rho) {
  z <- (x1^2 - 2 * rho * x1 * x2 + x2^2) / (1 - rho^2)
  (1 / (2 * pi * sqrt(1 - rho^2))) * exp(-z / 2)
}

grid_df$density <- bivariate_normal_density(grid_df$x1, grid_df$x2, rho = 0.6)
```

```
ggplot() +
  geom_point(
    data = as_tibble(bivariate_sample), aes(x = x1, y = x2),
    alpha = 0.15, size = 0.6, colour = "steelblue"
  ) +
  geom_contour(
    data = grid_df, aes(x = x1, y = x2, z = density),
    colour = "darkred", breaks = c(0.01, 0.05, 0.1)
  ) +
  labs(
    title = "Simulated bivariate Normal (rho = 0.6) with theoretical density contours",
    x = expression(X[1]), y = expression(X[2])
  ) +
  theme_minimal(base_size = 11)
```

Simulated bivariate Normal (rho = 0.6) with theoretical density contours

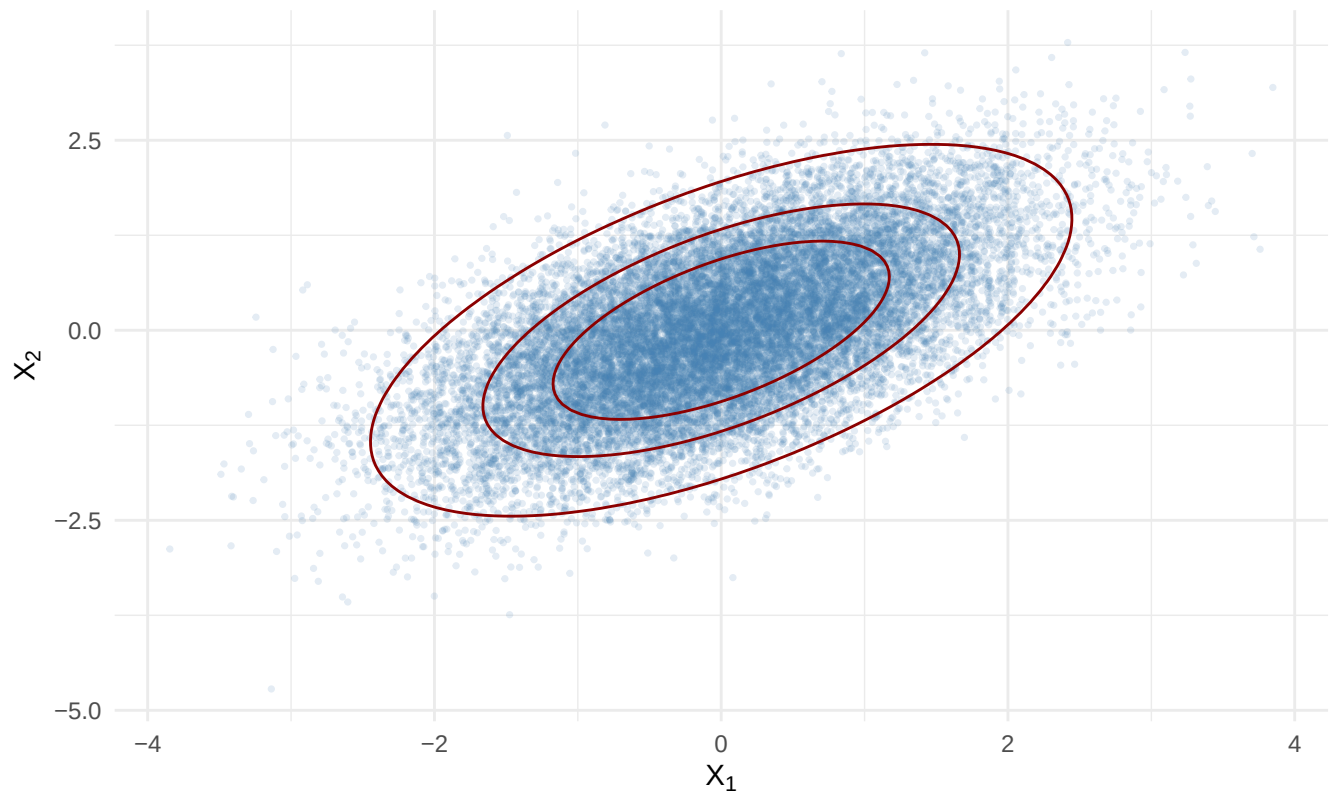


Figure 2: Simulated bivariate Normal pairs with rho = 0.6, overlaid with theoretical 50%, 90%, and 99% density contours.

```
trivariate_df <- as_tibble(trivariate_sample, .name_repair = "unique") %>%
  rename(x1 = 1, x2 = 2, x3 = 3)

p_12 <- ggplot(trivariate_df, aes(x = x1, y = x2)) +
  geom_point(alpha = 0.15, size = 0.5, colour = "steelblue") +
  labs(title = "X1 vs X2 (rho = 0.5)") +
  theme_minimal(base_size = 10)
```

```
p_13 <- ggplot(trivariate_df, aes(x = x1, y = x3)) +
  geom_point(alpha = 0.15, size = 0.5, colour = "darkgreen") +
  labs(title = "X1 vs X3 (rho = 0.3)") +
  theme_minimal(base_size = 10)

p_23 <- ggplot(trivariate_df, aes(x = x2, y = x3)) +
  geom_point(alpha = 0.15, size = 0.5, colour = "purple") +
  labs(title = "X2 vs X3 (rho = 0.4)") +
  theme_minimal(base_size = 10)

p_12 + p_13 + p_23
```

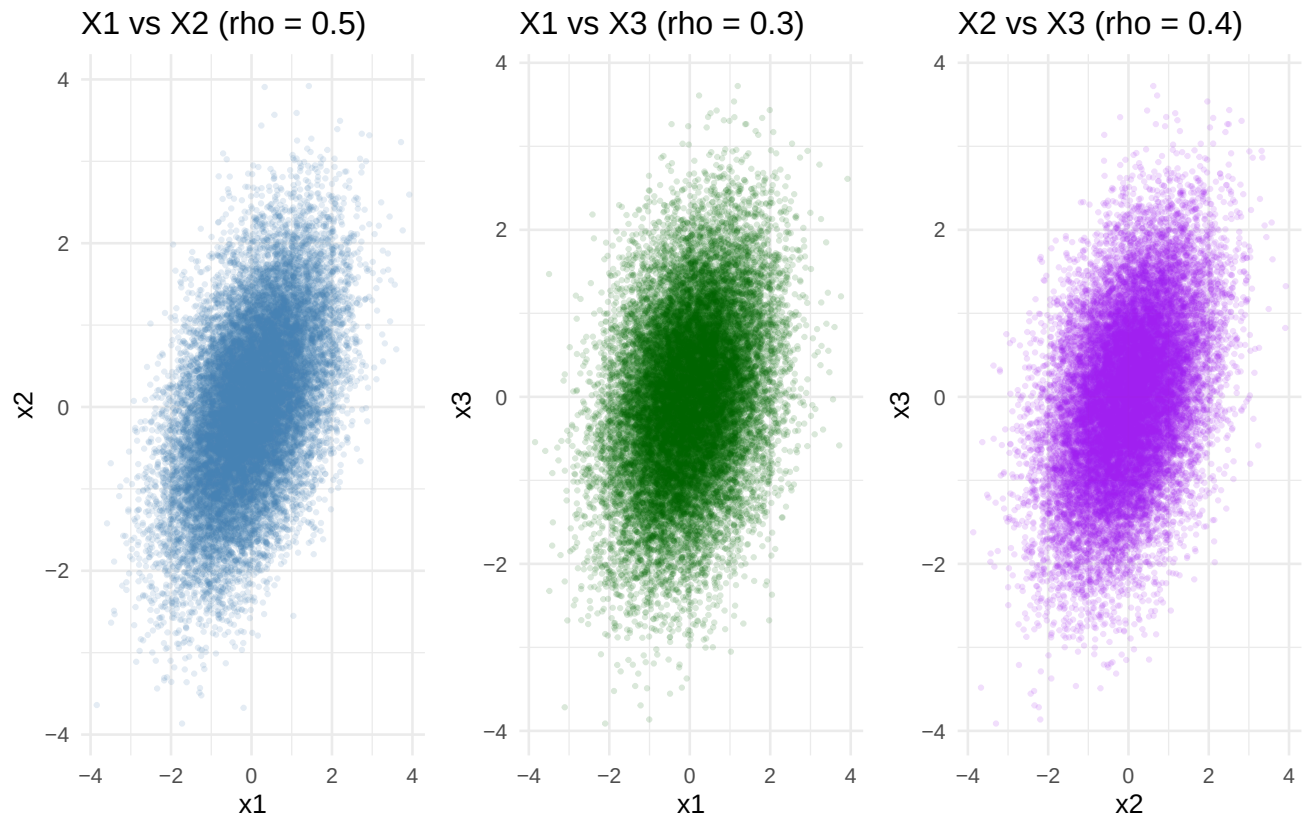


Figure 3: Pairwise scatterplots of the simulated trivariate Normal sample, illustrating the three pairwise correlations of the target covariance matrix (0.5, 0.3, 0.4).

```
ggplot(as_tibble(exp_uniform_sample), aes(x = x, y = y)) +
  geom_point(alpha = 0.15, size = 0.5, colour = "darkorange") +
  geom_abline(slope = 1, intercept = 0, colour = "black", linetype = "dashed") +
  coord_cartesian(xlim = c(0, 5), ylim = c(0, 5)) +
  labs(title = "Exponential(1) with conditional Uniform(0, X)", x = "X", y = "Y") +
  theme_minimal(base_size = 11)
```

## Numerical Examples

### Example 1: Bivariate Normal via the conditional method.

*Problem.* With  $\rho = 0.6$ , and draws  $w_1 = 0.5$ ,  $w_2 = 0.5$ , compute the resulting pair  $(x_1, x_2)$ .

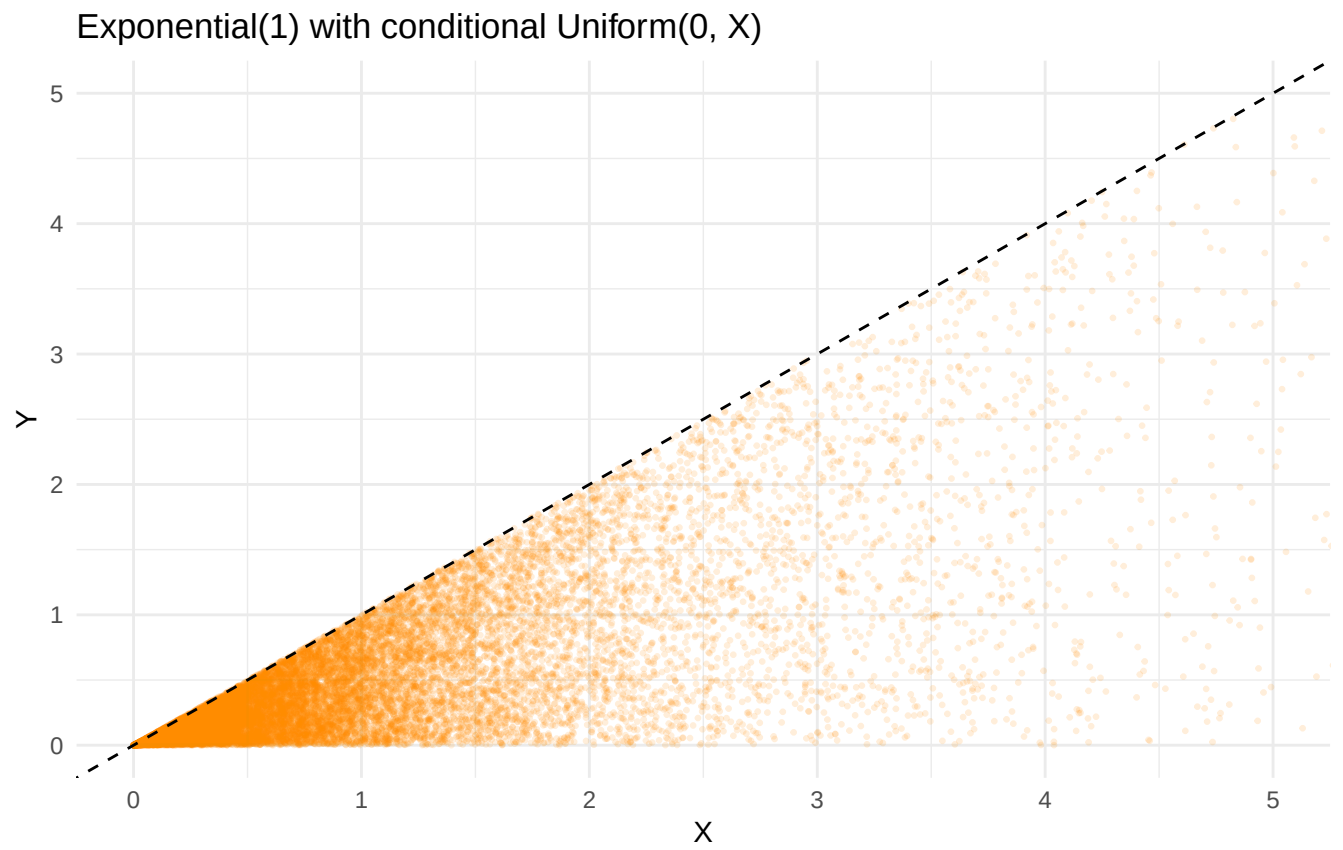


Figure 4: Simulated  $(X, Y)$  pairs from  $X \sim \text{Exponential}(1)$ ,  $Y | X \sim \text{Uniform}(0, X)$ ; every point lies below the line  $y = x$  by construction.

*Solution.*  $x_1 = w_1 = 0.5$ , and

$$x_2 = \rho w_1 + \sqrt{1 - \rho^2} w_2 = 0.6(0.5) + \sqrt{1 - 0.36}(0.5) = 0.3 + (0.8)(0.5) = 0.3 + 0.4 = 0.7.$$

*Interpretation.* The pair  $(0.5, 0.7)$  shows the expected positive association: because  $\rho = 0.6 > 0$ , a positive value of  $x_1$  pulls the generated  $x_2$  toward positive values as well, though the independent component  $w_2$  still contributes its own variability.

### Example 2: Trivariate Normal via the Cholesky method.

*Problem.* Using the target covariance matrix of Section 10.2, with Cholesky factor  $\mathbf{L}$  (computed in R above) and standard Normal draws  $\mathbf{z} = (0.2, -0.1, 0.5)'$ , compute  $\mathbf{x} = \mathbf{L}\mathbf{z}$  (the mean vector is  $\mathbf{0}$ ).

*Solution.* Row by row,

$$\begin{aligned} x_1 &= L_{11}z_1 = 1.000(0.2) = 0.2000, \\ x_2 &= L_{21}z_1 + L_{22}z_2 = 0.500(0.2) + 0.866(-0.1) = 0.100 - 0.0866 = 0.0134, \\ x_3 &= L_{31}z_1 + L_{32}z_2 + L_{33}z_3 = 0.300(0.2) + 0.289(-0.1) + 0.909(0.5) \\ &= 0.060 - 0.0289 + 0.4546 = 0.4857. \end{aligned}$$

*Interpretation.* The lower-triangular structure of  $\mathbf{L}$  means each coordinate is built from the standard Normal draws up to and including its own index, exactly mirroring the sequential conditional construction of Section 7:  $X_1$  depends only on  $Z_1$ ,  $X_2$  depends on  $Z_1$  and  $Z_2$ , and  $X_3$  depends on all three.

### Example 3: The Exponential-conditional-Uniform bivariate construction.

*Problem.* With rate 1 for the Exponential marginal, and draws  $u_1 = 0.3935$  (for  $X$ ) and  $u_2 = 0.4$  (for  $Y \mid X$ ), compute the resulting pair.

*Solution.* By inverse transformation,  $x = -\ln(1 - u_1) = -\ln(0.6065) = 0.5000$ . Given  $x = 0.5$ ,  $Y \sim \text{Uniform}(0, 0.5)$ , so  $y = u_2 \cdot x = 0.4 \times 0.5 = 0.2000$ .

*Interpretation.* By construction,  $Y$  can never exceed  $X$ , which is exactly the triangular region visible in the scatterplot above; this bivariate structure arises naturally, for instance, as the joint distribution of a component's total lifetime ( $X$ ) and the time of a uniformly-timed random inspection that occurs sometime during that lifetime ( $Y$ ).

### Example 4: A discrete conditional construction (Poisson thinning).

*Problem.* Given a realized count  $X = 4$ , and independent Uniform draws  $u = (0.3, 0.7, 0.2, 0.9)$  used to construct  $Y \mid X = 4 \sim \text{Binomial}(4, 0.5)$  as a sum of four Bernoulli(0.5) trials (success if the corresponding  $u_i \leq 0.5$ ), determine  $Y$ .

*Solution.* Successes occur for  $u_1 = 0.3$  and  $u_3 = 0.2$  (both  $\leq 0.5$ ), and failures for  $u_2 = 0.7$  and  $u_4 = 0.9$ . Hence  $Y = 1 + 0 + 1 + 0 = 2$ .

*Interpretation.* This construction, in which each of  $X$  “events” is independently retained with probability  $p$ , is known as **Poisson thinning** when  $X$  itself is Poisson; it is used to model, for example, the number of defective items among a random number of items produced, and is a discrete illustration of exactly the same sequential conditional principle used for the continuous examples above.

## Applications

- **Statistics.** Multivariate analysis of variance, multivariate regression diagnostics, and copula-based dependence modelling all require simulating correlated random vectors for validation and power studies.
- **Machine Learning.** Generating correlated synthetic features for benchmarking algorithms, and simulating from Gaussian graphical models, both rely directly on the methods of this unit.

- **Data Science.** Producing realistic synthetic multivariate datasets that preserve a specified correlation structure, for privacy-preserving data sharing or for stress-testing analytics pipelines.
- **Finance.** Simulating correlated asset returns via the Cholesky method is the standard first step in portfolio value-at-risk estimation and in Monte Carlo pricing of multi-asset derivatives.
- **Reliability Engineering.** Simulating dependent component lifetimes, when components share a common operating environment or a common manufacturing batch, requires exactly the multivariate techniques of this unit rather than independent univariate generation.
- **Monte Carlo Methods.** Variance reduction techniques such as antithetic and common random number methods often require deliberately correlated, rather than independent, simulated vectors.
- **Bayesian Statistics.** Direct simulation from a multivariate posterior via the sequential conditional method, when the necessary conditionals are tractable, and Gibbs sampling (Remarks to Theorem 6.1), when only full conditionals are available, are both applications of the chain-rule idea developed in this unit.
- **Operations Research.** Simulating correlated demand across multiple products or locations in inventory and supply chain models.
- **Engineering.** Simulating correlated structural loads (e.g. wind and seismic loads acting jointly on a structure) for reliability- based design.

## Advantages

- The sequential conditional method is completely general: it applies to any joint distribution for which the required conditional densities can be derived and simulated, continuous, discrete, or mixed.
- The Cholesky method is exact, numerically stable, and computationally cheap relative to the dimension of the problem, making it the standard tool for Gaussian (and Gaussian-copula) multivariate simulation.
- Both methods are exact, in the same sense as the univariate methods of earlier units: no asymptotic or iterative approximation is involved.

## Limitations

- The sequential conditional method requires closed-form (or at least efficiently simulable) conditional distributions at every stage; for many multivariate targets, deriving these conditionals is itself a difficult problem, which is precisely the situation that motivates Gibbs sampling and other Markov chain Monte Carlo methods.
- The Cholesky method, as presented, is specific to the (multivariate) Normal distribution; constructing a non-Normal random vector with prescribed marginals and a prescribed correlation structure requires additional machinery, such as copula methods, that lie beyond the scope of this unit.
- The one-time cost of the Cholesky decomposition itself is  $O(p^3)$ , which can become a computational bottleneck for very high-dimensional covariance matrices, such as those arising in spatial statistics with thousands of locations.

## Practical Tips

- **Always validate that a covariance matrix is symmetric positive definite before decomposing it.** A matrix built by hand from individually plausible pairwise correlations can fail to be jointly consistent (positive definite); R's `chol()` raises an error in this case, which should be treated as a genuine modelling warning rather than worked around.
- **Remember that ordering does not affect correctness, but does affect tractability, in the sequential conditional method.** If the natural ordering of coordinates leads to an intractable conditional distribution, consider whether a different ordering yields simpler conditionals for the same joint target.
- **Use `chol()`'s return convention correctly.** R's `chol()` returns an *upper* triangular matrix  $\mathbf{U}$  with  $\mathbf{U}'\mathbf{U} = \Sigma$ ; the lower-triangular  $\mathbf{L}$  of Theorem 6.2 is obtained as `t(chol(Sigma))`, and confusing the

two conventions is a common source of silent transposition errors.

- **Do not confuse the sequential conditional method with Gibbs sampling.** The former is a single-pass exact algorithm; the latter is an iterative, only asymptotically exact Markov chain algorithm, used precisely when the former's requirements cannot be met.
- **Vectorize the Cholesky-based generator.** As implemented in `generate_multivariate_normal()`, generating the entire matrix of standard Normal draws at once and applying a single matrix multiplication is far faster in R than looping over replications.

## Summary

- Generating a random vector generally requires an explicit mechanism for dependence between coordinates; generating each coordinate independently destroys any dependence structure entirely.
- The chain rule factors any joint density into a marginal density and a sequence of conditional densities; Theorem 6.1 proves that drawing coordinates sequentially from this factorization reproduces the exact joint distribution, for any valid ordering of coordinates.
- For the multivariate Normal distribution, the Cholesky decomposition of the covariance matrix gives a direct, single-step linear transformation of independent standard Normal variables that reproduces the exact target distribution (Theorem 6.2), and this construction is shown to coincide exactly with the sequential conditional method specialized to Normal conditionals.
- The sequential conditional method should be distinguished carefully from Gibbs sampling, an iterative approximate technique used when the successive conditionals required by the chain rule are not available in closed form.

## Key Formulae

- Chain rule factorization:  $f(x_1, \dots, x_p) = f_1(x_1) \prod_{i=2}^p f_i(x_i \mid x_1, \dots, x_{i-1})$ .
- Multivariate Normal moment generating function:  $M_{\mathbf{X}}(\mathbf{t}) = \exp(\mathbf{t}'\boldsymbol{\mu} + \frac{1}{2}\mathbf{t}'\boldsymbol{\Sigma}\mathbf{t})$ .
- Cholesky construction:  $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}'$ ,  $\mathbf{X} = \boldsymbol{\mu} + \mathbf{L}\mathbf{Z}$ ,  $\mathbf{Z} \sim N_p(\mathbf{0}, \mathbf{I}_p)$ .
- Bivariate Normal blend ( $p = 2$ ):  $X_1 = W_1$ ,  $X_2 = \rho W_1 + \sqrt{1 - \rho^2} W_2$ .

## Key Terms

- **Bivariate distribution.** The joint distribution of two random variables considered together.
- **Chain rule (multiplication rule).** The factorization of a joint density into a marginal density and a sequence of conditional densities.
- **Cholesky decomposition.** The factorization  $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}'$  of a symmetric positive definite matrix into a lower triangular factor and its transpose.
- **Conditional density.** The density of one random variable given the observed values of others.
- **Gibbs sampling.** An iterative Markov chain Monte Carlo method that cycles through full conditional distributions; distinguished from the exact, single-pass sequential conditional method of this unit.
- **Multivariate Normal distribution.** The joint distribution of a random vector whose moment generating function has the exponential quadratic form (6.2).
- **Random vector.** A collection of random variables considered jointly, with a specified joint distribution.
- **Sequential conditional method.** Generating a random vector one coordinate at a time, each conditional on the coordinates already generated, following the chain rule factorization.

## Exercises

### Multiple Choice Questions

1. The chain rule factorizes a joint density into
  - (a) a product of independent marginal densities

- (b) a marginal density and a sequence of conditional densities
  - (c) a sum of conditional densities
  - (d) a ratio of two marginal densities
2. According to Theorem 6.1, the ordering of coordinates in the sequential conditional method
    - (a) must always be the natural numerical order
    - (b) does not affect correctness, though it may affect tractability
    - (c) must be chosen to minimize variance
    - (d) is irrelevant to both correctness and tractability
  3. The Cholesky decomposition factors a covariance matrix as
    - (a)  $\Sigma = \mathbf{L} + \mathbf{L}'$
    - (b)  $\Sigma = \mathbf{L}\mathbf{L}'$
    - (c)  $\Sigma = \mathbf{L}^{-1}$
    - (d)  $\Sigma = \mathbf{L}\mathbf{L}$
  4. In the bivariate Normal blend  $X_2 = \rho W_1 + \sqrt{1 - \rho^2} W_2$ , the role of  $\sqrt{1 - \rho^2}$  is to
    - (a) shift the mean of  $X_2$
    - (b) ensure  $X_2$  has variance 1
    - (c) ensure  $X_1$  and  $X_2$  are independent
    - (d) ensure  $\rho$  lies in  $(-1, 1)$
  5. Gibbs sampling differs from the sequential conditional method of this unit primarily because Gibbs sampling
    - (a) is exact after a single pass
    - (b) uses full conditionals and iterates, converging only asymptotically
    - (c) does not require any conditional distributions
    - (d) only applies to discrete distributions
  6. The one-time cost of the Cholesky decomposition of a  $p \times p$  covariance matrix is
    - (a)  $O(p)$
    - (b)  $O(p^2)$
    - (c)  $O(p^3)$
    - (d)  $O(\log p)$
  7. R's `chol()` function returns
    - (a) the lower triangular factor  $\mathbf{L}$  directly
    - (b) the upper triangular factor  $\mathbf{U}$  with  $\mathbf{U}'\mathbf{U} = \Sigma$
    - (c) the inverse of the covariance matrix
    - (d) the eigenvalues of the covariance matrix
  8. In the Exponential-conditional-Uniform construction of this unit, the generated pair  $(X, Y)$  always satisfies
    - (a)  $Y \geq X$
    - (b)  $Y \leq X$
    - (c)  $Y = X$
    - (d) no fixed relationship
  9. A necessary condition for the Cholesky decomposition of  $\Sigma$  to exist is that  $\Sigma$  be
    - (a) diagonal
    - (b) symmetric positive definite
    - (c) upper triangular
    - (d) a correlation matrix specifically
  10. Poisson thinning, as illustrated in this unit, is an example of
    - (a) the Cholesky method
    - (b) a discrete sequential conditional construction
    - (c) the inverse transformation method applied to a continuous distribution
    - (d) acceptance-rejection

**Answers.** 1(b), 2(b), 3(b), 4(b), 5(b), 6(c), 7(b), 8(b), 9(b), 10(b).

## Short Answer Questions

1. State the chain rule factorization of a joint density for  $p = 3$  coordinates.
2. Explain why generating each coordinate of a random vector independently generally fails to reproduce a specified joint distribution.
3. Explain, in your own words, why the ordering of coordinates does not affect the correctness of the sequential conditional method.
4. State the bivariate Normal blending formula and identify the roles of  $\rho$  and  $\sqrt{1 - \rho^2}$ .
5. Explain what it means for a matrix to be symmetric positive definite, and why this property is required for the Cholesky decomposition to exist.
6. Distinguish the sequential conditional method from Gibbs sampling in one or two sentences.
7. Why must R's `chol()` output be transposed before it can be used directly as the  $\mathbf{L}$  of Theorem 6.2?
8. Explain, using Example 3, why  $Y \leq X$  always holds in the Exponential-conditional-Uniform construction.
9. Give one application area, distinct from those emphasized in this unit, where correlated random vector simulation is essential.
10. Explain why the computational cost of the Cholesky method has two distinct components with different growth rates in  $p$  and in  $n$ .

## Long Answer Questions

1. State and prove Theorem 6.1 (correctness of the sequential conditional method) by induction, and explain why the base case and inductive step together establish the result for general  $p$ .
2. State and prove Theorem 6.2 (the Cholesky construction of the multivariate Normal), using the moment generating function argument in full.
3. Derive the equivalence between the sequential conditional method and the Cholesky method for the Normal distribution, as presented in the Mathematical Derivations section, and explain the role of the triangular structure of  $\mathbf{L}$ .
4. Derive the bivariate Normal blending construction  $X_2 = \rho W_1 + \sqrt{1 - \rho^2} W_2$  from first principles, verifying both the variance and the correlation of the resulting pair.
5. Compare the sequential conditional method and the Cholesky method in terms of generality, computational cost, and the class of distributions each can generate.
6. Discuss the distinction between the sequential conditional method and Gibbs sampling in detail, including the type of conditional distribution each requires and the exactness (or lack thereof) of each method.
7. Explain, with reference to a concrete example, why a covariance matrix assembled from individually valid pairwise correlations can fail to be positive definite, and describe how this failure would be detected in practice.
8. Discuss at least two applications of multivariate random variable generation in finance or reliability engineering, explaining specifically how correlation between coordinates affects the quantity of ultimate interest in each application.
9. Explain how the Exponential-conditional-Uniform construction of this unit could be adapted to model the joint distribution of a component's lifetime and the time of a randomly timed inspection, and discuss what modification would be needed if the inspection time were not uniformly distributed over the lifetime.
10. Discuss the computational and numerical considerations that arise when extending the Cholesky method to high-dimensional covariance matrices, and describe at least one practical mitigation.

## Programming Exercises

1. Implement `generate_bivariate_normal()` from first principles (already provided) and reproduce the manual calculation of Example 1 by fixing the random seed appropriately.
2. Implement `generate_multivariate_normal()` from first principles (already provided) and reproduce the manual calculation of Example 2 by fixing the random seed and manually extracting the standard Normal draws used.

3. Write an R function that checks whether a supplied matrix is symmetric positive definite (using `chol()` inside a `tryCatch()`), and test it on both a valid and an invalid covariance matrix.
4. Using `MASS::mvrnorm()` or `mvtnorm::rmvnorm()` as a benchmark, verify that `generate_multivariate_normal()` produces samples with statistically indistinguishable covariance structure for a 4-dimensional target of your choosing.
5. Implement `generate_exponential_uniform_pair()` from first principles (already provided) and empirically verify that the conditional distribution of  $Y$  given  $X$  in a narrow bin around  $x = 1$  is approximately  $\text{Uniform}(0, 1)$ .
6. Write an R function implementing the sequential conditional method for a three-stage discrete Markov-chain-like construction:  $X_1 \sim \text{Bernoulli}(0.5)$ ,  $X_2 | X_1 \sim \text{Bernoulli}(0.7 \text{ if } X_1 = 1 \text{ else } 0.2)$ ,  $X_3 | X_2 \sim \text{Bernoulli}(0.7 \text{ if } X_2 = 1 \text{ else } 0.2)$ , and estimate  $P(X_3 = 1)$  from a large simulated sample.
7. Reproduce the trivariate pairwise scatterplot figure of this unit for a covariance matrix of your own choosing, and verify the empirical correlations against the target values.
8. Write a function that generates from a bivariate distribution with Exponential marginals and a specified dependence via the composition idea of Unit 3.5 combined with the conditional method of this unit (for example, a shared-shock construction), and examine the resulting correlation empirically.
9. Benchmark, using `system.time()`, the Cholesky method against a naive coordinate-by-coordinate conditional-regression implementation for simulating a 10-dimensional Normal vector, and discuss the results.
10. Implement Poisson thinning in R: simulate  $X \sim \text{Poisson}(\lambda)$ , then  $Y | X = x \sim \text{Binomial}(x, p)$ , and verify empirically that the marginal distribution of  $Y$  is  $\text{Poisson}(\lambda p)$ , a classical result in point process theory.

## Mini Projects

1. **Simulating correlated asset returns.** Using the Cholesky method, simulate ten years of daily returns for four correlated assets with a covariance matrix estimated from (or plausibly specified for) real or illustrative data. Compute the simulated portfolio's value-at-risk at the 95% and 99% confidence levels for an equally weighted portfolio, and compare these figures to those obtained if the assets were (incorrectly) simulated as independent. Discuss, quantitatively, how ignoring correlation distorts the risk assessment.
2. **From sequential conditionals to Gibbs sampling.** Implement the exact sequential conditional method for a bivariate Normal target with a known correlation  $\rho$  (as in Algorithm 6.2), and separately implement a Gibbs sampler for the same target using its full conditional distributions  $X_1 | X_2 = x_2 \sim N(\rho x_2, 1 - \rho^2)$  and  $X_2 | X_1 = x_1 \sim N(\rho x_1, 1 - \rho^2)$ , iterated for a large number of steps. Compare the two resulting samples graphically and using summary statistics, discuss the burn-in behaviour of the Gibbs sampler, and explain why the exact method is preferable whenever it is available.

**Difficulty labels.** Multiple choice, short answer: **Easy**. Long answer questions 3, 4, 7, 8, 9, programming exercises 1-6: **Medium**. Long answer questions 1, 2, 5, 6, 10, programming exercises 7-10: **Medium to Advanced**. Mini projects: **Advanced**.

## References

- Casella, G., & Berger, R. L. (2002). *Statistical inference* (2nd ed.). Duxbury.
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations* (4th ed.). Johns Hopkins University Press.
- Pourahmadi, M. (1999). Joint mean-covariance models with applications to longitudinal data: Unconstrained parameterisation. *Biometrika*, 86(3), 677-690.
- Ripley, B. D. (1987). *Stochastic simulation*. Wiley.
- Rizzo, M. L. (2019). *Statistical computing with R* (2nd ed.). CRC Press.
- Ross, S. M. (2020). *Simulation* (6th ed.). Academic Press.